Complex does have its own feature of the <=> operator. A comparison involving a complex number with an imaginary part that is not zero will return nil. In other cases it will compare the real part of the complex number with the other operand. The == method, however will compare complex numbers regardless of their imaginary dimension.

The conversion methods to_c, to_d, to_f, to_i, to_json, to_r, and to_s are defined. The numeric methods will return a RangeError if the imaginary part is not integer 0.

The to_json method returns a JSON representation of the hash {"json_class" => "Complex", "r" => self.real, "i" => self.imaginary}. (See *Chapter 28, Library Reference: I/O* for more information.) The JSON string can be deserialized with the class method Complex.json_create.

BigDecimal

Ruby has a lot of literal numeric types: integers, floating-point numbers, rational numbers, and complex numbers. There's one common need that is not filled by Ruby's literals and which you need the standard library for: precise representation of decimal values.

What do we mean by precision? Well, here's an example that you can replicate in your own IRB session:

```
1.1 - 0.8 # => 0.30000000000000004
```

That seems…wrong? Your fourth grade math teacher would probably be appalled.

It is important that you are familiar with two important facts:

- Floating-point numbers are inherently imprecise—it's not just a Ruby issue—and shouldn't be used for values like money, where exact precision is necessary. (Floating-point numbers are used because they are smaller in memory and faster in calculation.)
- When you do need a precise number in Ruby, use BigDecimal.

Floating-point numbers are imprecise because they are trying to represent base-10 decimals in a base-2 format, and there just isn't a one-to-one match.

Internally, floating-point numbers are represented as a set of bytes (64 on most of the machines you'll be using) of which some are used for a base number and the rest are used for an exponent, so 24.68 might be represented as 2468 x 10**-2. Without getting too deep into the exact details of the representation, this also means that, as the floating-point numbers get farther from zero, the worse the floating-point standard gets at matching floats to actual numbers. One implication is that floating-point number errors are worse if the two operands of an operation are of vastly different magnitudes.

When you need precise decimal values, Ruby provides the BigDecimal class. Internally, BigDecimal uses a decimal format that allows you to specify the number of digits used, and then represents that value exactly in any future math operations you might perform.

To use BigDecimal, you must require bigdecimal in your application. Once you've required bigdecimal, creating new instances of BigDecimal is a little odd. The BigDecimal class doesn't have a constructor; instead you use a Kernel method, also called BigDecimal, which makes it look like you are using one of the conversion methods:

```
require "bigdecimal"
BigDecimal("3.14")  # => 0.314e1
BigDecimal(4.2, 2)  # => 0.42e1
```